

Dashboard Marketing

Expansão CDC — Clínica da Cidade

Projeto	Dashboard Marketing — Expansão CDC
Tipo	Aplicação Web + Pipeline de Dados
Linguagens	Python 3 + HTML5 + CSS3 + JavaScript (ES6)
Frontend	HTML / CSS / JavaScript puro + Chart.js
Backend	Python 3 (scripts de coleta e agregação)
Hospedagem	GitHub Pages (estático)
Repositório	github.com/geovane-Maze/dashboard-marketing
URL	geovane-maze.github.io/dashboard-marketing/
Data documento	15/05/2026

1. Visão Geral do Projeto

O **Dashboard Marketing** é uma aplicação web responsiva desenvolvida para consolidar e visualizar, em um único painel, todos os indicadores de marketing e vendas da expansão da rede **Clínica da Cidade (CDC)**. O sistema integra automaticamente dados de cinco fontes distintas — RD Station Marketing, RD Station CRM, Google Ads, Meta Ads e Google Analytics 4 — e os apresenta em 9 páginas distintas com KPIs, gráficos interativos, filtros por período e campanha, e detalhamento por criativo.

O objetivo principal é fornecer ao time de marketing visibilidade em tempo quase real sobre captação de leads (orgânicos e pagos), performance de campanhas, custo por lead (CPL), funil de vendas no CRM, e atribuição por canal — sem depender de ferramentas pagas de BI.

Páginas do Dashboard

- **Visão Executiva** — KPIs principais, alertas e resumo consolidado
- **Evolução Mensal** — gráficos com últimos 4 meses (com filtro local)
- **Funil CRM** — pipeline de vendas com etapas ativas e taxas de conversão
- **Gargalos & Perdas** — análise de motivos de perda de negócios
- **Meta Ads** — performance de campanhas Facebook/Instagram
- **Google Ads** — performance de campanhas Google (incluindo Performance Max)
- **Criativos** — top criativos Meta Ads com filtros de período, campanha e status
- **UTM / Atribuição** — leads por source, medium, campaign e canal
- **Google Analytics 4** — métricas de tráfego do site

2. Linguagens e Tecnologias

2.1 Stack Tecnológico

O projeto foi construído com tecnologias **nativas e gratuitas**, sem depender de frameworks pesados, o que torna a manutenção simples e o deploy extremamente leve.

Camada	Linguagem	Tecnologia / Biblioteca
Coleta de Dados	Python 3	requests, gspread, google-auth, facebook-business
Pipeline / Agregação	Python 3	gspread, pandas, json (stdlib)
Frontend	JavaScript (ES6)	JavaScript puro (vanilla), sem frameworks
Visualização	JavaScript	Chart.js 4.4 (gráficos)
Estilização	CSS3	CSS Grid + Flexbox + variáveis CSS
Estrutura	HTML5	HTML5 semântico
Armazenamento	JSON	Arquivo summary.json (gerado pelo Python)
Hospedagem	GitHub Pages	Servidor estático gratuito
Versionamento	Git / GitHub	Controle de versão completo

2.2 Por que essas escolhas?

Python para o backend — Linguagem ideal para integrações com APIs, manipulação de dados e automação. Possui SDKs oficiais para Google (gsread, google-ads, google-analytics-data) e Meta (facebook-business).

JavaScript vanilla no frontend — Decisão consciente para evitar complexidade de build (sem Webpack, sem React, sem Node.js). O dashboard carrega instantaneamente, é fácil de manter e roda em qualquer navegador moderno sem etapa de compilação.

Chart.js — Biblioteca de gráficos leve (~80KB), com excelente qualidade visual, suporte a responsividade e API simples. Roda direto do CDN.

JSON como camada de dados — O Python gera um único arquivo *summary.json* consolidado, que o frontend lê em uma única requisição. Isso elimina a necessidade de um banco de dados ou API server-side.

2.3 Bibliotecas Python (requirements.txt)

```
requests # chamadas HTTP às APIs gspread # leitura/escrita no Google Sheets
google-auth # autenticação Google (Service Account) google-auth-oauthlib # OAuth
para tokens GA4 / Google Ads google-analytics-data # SDK GA4 google-ads # SDK
Google Ads facebook-business # SDK Meta Ads python-dotenv # carregar variáveis do
.env pandas # manipulação tabular (writer de planilhas)
```

3. Arquitetura do Projeto

O sistema funciona em três camadas independentes que se comunicam por meio de uma planilha Google Sheets central (camada de persistência) e um arquivo JSON (camada de leitura pelo frontend):

#	Camada	Função
1	Coletores (Python)	Buscam dados das APIs externas e gravam na planilha
2	Pipeline (Python)	Lê planilha, agrega e gera o summary.json
3	Dashboard (Frontend)	Lê o summary.json e renderiza gráficos/KPIs

3.1 Fluxo de Dados

```
[APIs Externas] | RD Station Marketing (api.rd.services) | RD Station CRM (crm.rdstation.com) | Google Ads (planilha de export) | Meta Ads (planilha de export) | Google Analytics 4 (analyticsdata.googleapis.com) v [main.py] --grava--> [Google Sheets (planilha central)] | v [generate_dashboard_data.py] --le e agrega--> [summary.json] | v [git push origin main] --deploy--> [GitHub Pages] | v [index.html + Chart.js] Renderiza dashboard
```

4. Estrutura de Arquivos

```
dashboard-marketing/ | |-- main.py # Orquestrador principal da coleta |-- config.py # Carrega variáveis do .env |-- requirements.txt # Dependências Python |-- .env # Tokens e credenciais (NUNCA versionado) |-- .env.example # Modelo de .env |-- index.html # Redireciona para /dashboard | |-- collectors/ # Módulos de coleta (1 por fonte) | |-- rdstation.py # RD Station Marketing (leads) | |-- rdcrm.py # RD Station CRM (deals/atividades) | |-- google_ads.py # Google Ads via API oficial | |-- meta_ads.py # Meta Ads via API oficial | |-- ga4.py # Google Analytics 4 | |-- ads_sheets.py # Fallback: lê planilhas exportadas | |-- sheets/ | |-- writer.py # Grava nas abas da planilha central | |-- scripts/ | |-- generate_dashboard_data.py # Pipeline de agregação | |-- dashboard/ # FRONTEND (publicado no GitHub Pages) | |-- index.html # Dashboard (HTML + CSS + JS inline) | |-- data/ | |-- summary.json # Dados consolidados (gerado pelo pipeline) | |-- credentials/ # Service Account JSON (NUNCA versionado) | |-- gerar_token_rd.py # Gera refresh_token RD Station |-- gerar_token_ga4.py # Gera token OAuth do GA4 |-- gerar_token_google_ads.py # Gera refresh_token Google Ads
```

5. Coletores de Dados

Cada coletor é um módulo Python independente, responsável por uma fonte de dados. Todos seguem o mesmo padrão: autenticam, paginam (se aplicável), normalizam os dados e retornam uma lista de dicionários.

5.1 rdstation.py — RD Station Marketing

Coleta todos os contatos (leads) da plataforma RD Station Marketing. Endpoint base: `https://api.rd.services`

Etapas:

- Autentica via OAuth 2.0 com `refresh_token`, gerando `access_token`
- Lista segmentações (`/platform/segmentations`) e usa "Todos os contatos da base de Leads"
- Pagina contatos da segmentação (20 por página)
- Para cada contato: busca detalhes (`/platform/contacts/{uuid}`) e eventos de conversão
- Decodifica o campo `traffic_source` (Base64) para extrair UTMs
- Aplica dedup por UUID (RD pode repetir contatos entre páginas)
- Retorna lista com nome, email, telefone, cidade, estado, lifecycle, UTMs, canal/fonte (`cf_plug_opportunity_origin`), capital disponível, prazo de abertura, etc.

Robustez (fix aplicado em 14/05):

- **Retry exponencial** em 429/5xx (backoff 1s → 2s → 4s → 8s)
- **Refresh automático** de token em 401
- **Refresh proativo** a cada 100 contatos (evita expiração mid-run)
- **Sleep de 0,25s** entre chamadas (margem de rate limit)
- **Logging explícito** de falhas no fim do run

5.2 rdcrm.py — RD Station CRM

Coleta negócios (deals), atividades e tarefas da plataforma RD Station CRM. Endpoint base: `https://crm.rdstation.com/api/v1`

Coleta três tipos de entidades:

- **Deals** — oportunidades de venda com valor, estágio, responsável, motivo de perda
- **Atividades** — registros de interação (calls, e-mails, reuniões)
- **Tarefas** — to-dos e follow-ups

Autentica via API Key. Os deals alimentam o Funil CRM, a Visão Executiva (leads abertos/perdidos) e os gráficos de motivos de perda.

5.3 google_ads.py — Google Ads API

Coleta dados de campanhas Google Ads via API oficial. Atualmente em status de fallback (a API requer aprovação manual do `customer_id` pela conta). Quando indisponível, o sistema usa o coletor

ads_sheets.py como fallback.

5.4 meta_ads.py — Meta Ads API

Coleta dados de campanhas Meta (Facebook + Instagram) via Facebook Business SDK. Também opera com fallback para a planilha quando há erro de autenticação.

5.5 ga4.py — Google Analytics 4

Coleta métricas do GA4 (sessions, users, conversions, bounceRate, channels, devices, pages, sources) via google-analytics-data SDK. Autentica via OAuth 2.0.

5.6 ads_sheets.py — Fallback de Planilhas

Coletor de fallback que lê **planilhas exportadas manualmente** do Google Ads e Meta Ads (caso as APIs estejam indisponíveis). Detecta automaticamente dois formatos:

- **Formato DIÁRIO** — com coluna "Dia", gera dados granulares por data
- **Formato AGREGADO** — totais por campanha sem breakdown diário (preserva histórico)

6. Pipeline de Agregação

O arquivo `scripts/generate_dashboard_data.py` é o coração do sistema. Ele lê as abas brutas da planilha Google Sheets, aplica regras de negócio (qualificação, atribuição, normalização de etapas) e gera o **summary.json** que o dashboard consome.

Principais Funções

Função	O que produz
<code>aggregate_leads()</code>	monthly, daily, by_source, by_canal, by_lifecycle, by_capital, by_prazo
<code>aggregate_crm()</code>	active_funnel, total_active, total_won, total_lost, monthly_stages, monthly_won
<code>aggregate_meta_ads()</code>	monthly, daily, campaigns, campaign_monthly, creatives, total_gasto
<code>aggregate_google_ads()</code>	monthly, daily, campaigns, campaign_monthly, total_gasto
<code>aggregate_ga4()</code>	monthly, channels, devices, pages, sources
<code>aggregate_utm()</code>	sources, mediums, campaigns, *_monthly, *_daily

Regras de Negócio Importantes

- **Dedup defensivo** por id em leads e crm_deals (mesmo que a coleta já dedupe)
- **Normalização de etapas** CRM (Reunião PDF, Apresentação BP, Workshop, etc.)
- **Exclusão de etapas mortas** do funil ativo (Etapa Descarte, Sem etapa, ganhos/perdidos)
- **Classificação Repasse vs Expansão** de campanhas (keywords: taguatinga, maringá, praia grande)
- **Qualificação** de leads pelo lifecycle_stage (qualif ou mql)
- **Conversão de datas** para formato YYYY-MM (mês) e YYYY-MM-DD (dia)

7. Dashboard Frontend

Todo o dashboard está em um **único arquivo HTML** (~2.300 linhas), com CSS e JavaScript inline. Isso simplifica o deploy (nada para compilar) e garante carregamento rápido em qualquer dispositivo.

7.1 Tecnologias do Frontend

Tecnologia	Uso
HTML5	Estrutura semântica de 9 seções (uma por página)
CSS3	Layout responsivo (Grid + Flexbox), variáveis CSS para tema
JavaScript ES6+	Lógica de filtros, renderização e interatividade
Chart.js 4.4	Gráficos de barra, linha, pizza, donut e mistos
Fetch API	Carrega o summary.json no boot
LocalStorage	(não usado) — toda a sessão é stateless

7.2 Padrão de Renderização

O dashboard usa um padrão simples de **render por seção**. Quando o usuário navega para uma página, a função `render<NomeDaSecao>()` é chamada com base no filtro global. Funções principais:

- `renderExec()` — Visão Executiva
- `renderEvolucao()` — Evolução Mensal (filtro local)
- `renderCRM()` — Funil CRM
- `renderPerdas()` — Gargalos & Perdas
- `renderMeta()` — Meta Ads
- `renderGoogle()` — Google Ads
- `renderCreatives()` — Criativos
- `renderUTM()` — UTM / Atribuição
- `renderGA4()` — Google Analytics 4

7.3 Sistema de Filtros

- **Filtro global** (header) — período (De/Até) + campanha. Aplica em todas as páginas.
- **Filtro local Evolução Mensal** — De/Até em meses (independente do global, autoload últimos 4 meses).
- **Filtro local Criativos** — Período + Campanha (com tags Ativo/Pausado) + Status + Ordenação.
- **Atalhos rápidos** — Hoje, Últimos 7 dias, Este mês, Mês passado, Personalizado.

7.4 Mobile / Responsividade

Em telas $\leq 600\text{px}$ o dashboard troca a sidebar fixa por um **menu hambúrguer com drawer lateral**. KPIs viram cards empilhados, gráficos ocupam 100% da largura e tabelas têm scroll horizontal.

8. Configuração e Setup

8.1 Pré-requisitos

- Python 3.10+ instalado
- Conta Google com acesso à planilha central
- Service Account JSON (Google Cloud) com permissão Editor na planilha
- Refresh tokens válidos: RD Station Marketing, RD Station CRM, GA4, Google Ads
- Token de acesso Meta Business (caso queira usar API)

8.2 Arquivo .env

```
# Google
GOOGLE_CREDENTIALS_FILE=credentials/service-account.json
GOOGLE_SHEET_ID=<id_da_planilha_central> # RD Station Marketing
RD_CLIENT_ID=... RD_CLIENT_SECRET=... RD_REFRESH_TOKEN=... # RD Station CRM
RDCRM_TOKEN=... # GA4
GA4_PROPERTY_ID=... # Google Ads
GOOGLE_ADS_DEVELOPER_TOKEN=...
GOOGLE_ADS_CLIENT_ID=... GOOGLE_ADS_CLIENT_SECRET=...
GOOGLE_ADS_REFRESH_TOKEN=...
GOOGLE_ADS_CUSTOMER_ID=... # Meta Ads
META_ACCESS_TOKEN=... META_AD_ACCOUNT_ID=...
```

8.3 Instalação

```
# 1. Clone o repositório
git clone https://github.com/geovane-Maze/dashboard-marketing.git
cd dashboard-marketing # 2. Instale dependências Python
pip install -r requirements.txt # 3. Configure o .env (copie do .env.example)
cp .env.example .env # Edite o .env com seus tokens # 4. Coloque o service account JSON em credentials/
# 5. Rode o pipeline completo
python main.py # Coleta dados
python scripts/generate_dashboard_data.py # Gera summary.json
# 6. Commit e push para deploy
git add dashboard/data/summary.json
git commit -m 'update data'
git push origin main
```

9. Deploy via GitHub Pages

O dashboard é hospedado gratuitamente no **GitHub Pages**, servido a partir da pasta `/dashboard` do branch `main`. Qualquer push para o `main` que altere arquivos dentro de `/dashboard` dispara um deploy automático em aproximadamente 1 minuto.

Workflow de Atualização

```
Atualizar dados: python main.py (~3 minutos)
python scripts/generate_dashboard_data.py (~10 segundos)
git add dashboard/data/summary.json
git commit -m 'update <data>'
git push origin main
Aguardar ~1 min para o GitHub Pages atualizar.
Hard refresh no navegador: Ctrl+Shift+R
```

10. Manutenção e Operações Comuns

10.1 Atualizar Planilha do Google Ads

Quando o Google Ads envia um novo relatório de campanhas, atualiza-se `collectors/ads_sheets.py`:

```
GOOGLE_ADS_SHEET_ID = '<novo_id_da_planilha>'
GOOGLE_ADS_TAB_NAME = '<nome_exato_da_aba>'
```

Depois é só rodar o pipeline e fazer push. O sistema detecta automaticamente se o formato é diário ou agregado.

10.2 Tokens Expirados

- **RD Station Marketing** — rode `python gerar_token_rd.py`
- **GA4** — rode `python gerar_token_ga4.py`
- **Google Ads** — rode `python gerar_token_google_ads.py`

10.3 Resolução de Problemas Frequentes

Sintoma	Causa Provável	Solução
Dashboard não atualiza	GitHub Pages em cache	Ctrl+Shift+R no navegador
Leads com dados em branco	Rate limit RD Station	Já corrigido em 14/05 (retry exponencial)
Duplicatas em leads	API RD repete paginação	Já corrigido em 14/05 (dedup por UUID)
Google Ads desatualizado	Planilha em formato agregado	Solicitar export com coluna Dia
GA4 retorna 401	Token expirado	Rodar <code>gerar_token_ga4.py</code>

11. Histórico de Correções Críticas

Esta seção documenta correções importantes aplicadas ao longo do desenvolvimento, que valem ser conhecidas por quem for manter o projeto.

Data	Problema	Correção
13/05	Filtro global não funcionava nas páginas Meta/Google Ads	Adiciona filtro de período que controla o dropdown + tags Ativo/Pausado por campanha
13/05	Meta/Google Ads mostravam contagem errada de leads (vithas darplest daily) filtrado por source='metaads' / 'googlepcp' (fonte: RD Marketing)	Atualiza contagem Google Ads
14/05	Criativos: dropdown listava campanhas que não existiam	Atualiza contagem Google Ads
14/05	Lead duplicado (CARLOS DE SOUZA PEREIRA)	Atualiza contagem Google Ads
14/05	119 de 300 leads (40%) com telefone/cidade em branco	Atualiza contagem Google Ads
14/05	Google Ads: nova planilha em formato agregado	Atualiza contagem Google Ads

12. Snapshot de Métricas Atuais

Estado do dashboard em **15/05/2026** (última atualização do pipeline):

Fonte	Volume Coletado
Leads (RD Marketing)	306 contatos únicos
Deals (RD CRM)	679 negócios
Atividades CRM	126 registros
Tarefas CRM	489 registros
Meta Ads	14.865 linhas (dados de criativos por dia)
Google Ads	Histórico diário Jan-Mai 2026 preservado
Tamanho do summary.json	289 KB

13. Resumo Final

O **Dashboard Marketing Expansão CDC** é um projeto leve, robusto e totalmente gratuito de hospedar. Construído em **Python + JavaScript vanilla + Chart.js**, integra 5 fontes de dados em um único painel responsivo. Após as correções críticas de maio/2026, o sistema garante:

- **Zero duplicatas** de leads (dedup em 3 camadas)
- **Zero perda de dados** em coleta longa (retry exponencial + refresh de token)
- **Deteção automática** de formato de planilhas (diário vs agregado)
- **Filtros precisos** por dia exato em todas as páginas relevantes
- **Interface mobile** 100% funcional
- **Deploy gratuito** via GitHub Pages com atualização em ~1 minuto

Documento gerado automaticamente em 15/05/2026 às 11:02.